JavaScript

what is JavaScript?

JavaScript (JS) is a lightweight, high-level, interpreted programming language. It is one of the core technologies of the web, alongside HTML and CSS and is mainly used to make websites **interactive** and **dynamic**.

জাভাস্ক্রিপ্ট কী?

JavaScript (JS) একটি হালকা (lightweight), উচ্চস্তরের (high-level) ও ইন্টারপ্রেটেড (interpreted) প্রোগ্রামিং ভাষা। এটি HTML ও CSS-এর পাশাপাশি ওয়েবের অন্যতম মূল প্রযুক্তি এবং মূলত ওয়েবসাইটকে ইন্টারঅ্যাক্টিভ ও ডাইনামিক করতে ব্যবহৃত হয়।

JavaScript vs HTML: Static vs Interactive & Dynamic

HTML is static. It only defines the structure and content of a webpage, which stays the same unless manually reloaded.

JavaScript, on the other hand, makes a webpage interactive and dynamic. it can respond to user actions like clicks, typing, or scrolling, and update content in real time without reloading the page. In short, HTML builds the page, while JavaScript brings it to life.

HTML স্থির (static)। এটি শুধু ওয়েবপেজের গঠন ও কন্টেন্ট নির্ধারণ করে, যা manually reload না করলে পরিবর্তন হয় না।

অন্যদিকে, JavaScript ওয়েবপেজকে ইন্টারঅ্যাক্টিভ ও ডাইনামিক করে তোলে। এটি ইউজারের ক্লিক, টাইপিং বা স্ক্রলের মতো action-এ সাড়া দিতে পারে এবং পেজ রিলোড ছাড়াই রিয়েল-টাইমে কন্টেন্ট আপডেট করতে পারে। সংক্ষেপে বলতে গেলে, HTML পেজ তৈরি করে, আর JavaScript সেটাকে প্রাণবন্ত করে তোলে।

Interactive & Dynamic

Interactive (Responds to User Actions)

JavaScript makes a webpage respond to user actions like clicking, typing, hovering, or scrolling turning a fixed page into a live experience.

Interactive (ইউজারের অ্যাকশনে সাড়া দেয়)

JavaScript ওয়েবপেজকে ইউজারের ক্লিক, টাইপ, hover, বা স্ক্রলের মতো অ্যাকশনে সাড়া দিতে সাহায্য করে একটি স্থির পেজকে জীবন্ত অভিজ্ঞতায় রূপান্তরিত করে।

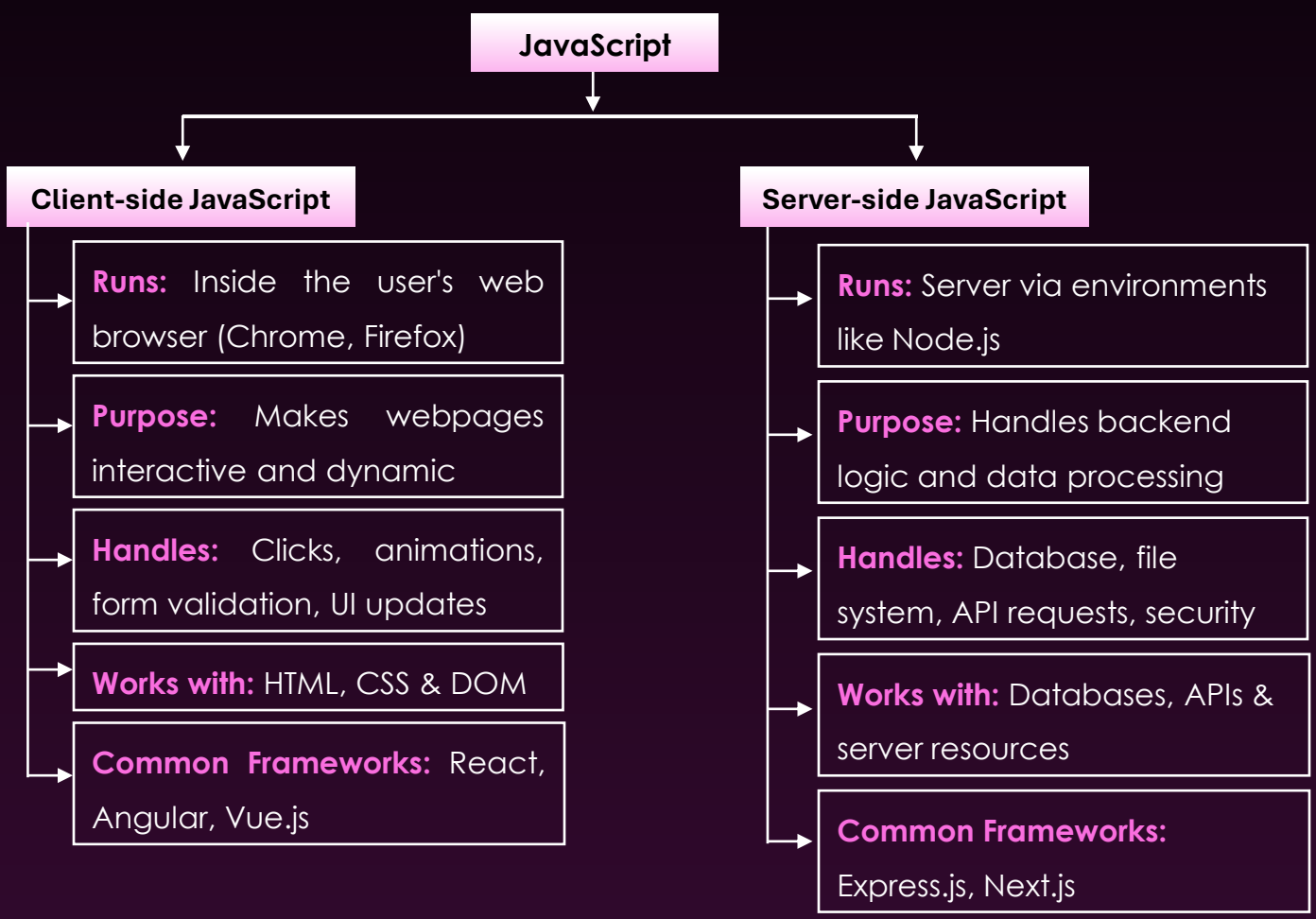
Dynamic (Updates Content in Real Time)

JavaScript can instantly change a page's text, style, or structure without reloading giving users a smooth, real-time experience.

Dynamic (রিয়েল-টাইমে কন্টেন্ট আপডেট হয়)

JavaScript পেজ রিলোড ছাড়াই পেজের টেক্সট, স্টাইল বা structure সাথে সাথে পরিবর্তন করতে পারে ইউজারকে দেয় একটি smooth, রিয়েল-টাইম অভিজ্ঞতা।

JavaScript Execution: Client-Side & Server-Side



How many types of JavaScript scripts are there?

There are three types of JavaScript based on how it's written in HTML.

1. Inline **2. internal and** **3. external.**

HTML-এ লেখার পদ্ধতির ভিত্তিতে JavaScript-কে তিন ভাগে ভাগ করা যায়:

১. inline **২. internal, এবং** **৩. external**

Inline JavaScript

JavaScript written directly inside an **HTML element's attribute**, such as onclick or onchange. It's placed right within the tag, without using a separate <script> block.

Inline JavaScript হলো সেই JavaScript যা সরাসরি কোনো HTML element-এর attribute-এর (যেমন onclick, onchange) ভেতরে লেখা হয়। এটি আলাদা <script> ব্লক ছাড়াই ট্যাগের ভেতরেই বসানো হয়।

```
<button onclick="alert('Hello!')">Click Me</button>
```

Internal JavaScript

JavaScript written inside a **<script> tag within the same HTML file**, usually placed in the <head> or before the closing </body> tag.

Internal JavaScript হলো সেই JavaScript যা একই HTML ফাইলের ভেতরে <script> ট্যাগে লেখা হয়, সাধারণত <head>-এ অথবা </body> ট্যাগের ঠিক আগে বসানো হয়।

```
<script>
  document.getElementById("myBtn").addEventListener("click", function() {
    alert("Hello!");
  });
</script>
</body>
```

External JavaScript

JavaScript written in a **separate .js file** and linked to the HTML file using the src attribute of the <script> tag.

বাংলা সংজ্ঞা: External JavaScript হলো সেই JavaScript যা একটি আলাদা .js ফাইলে লেখা হয় এবং <script> ট্যাগের src attribute ব্যবহার করে HTML ফাইলের সাথে link করা হয়।

```
<script src="script.js"></script>
</body>
```

JavaScript Data Types

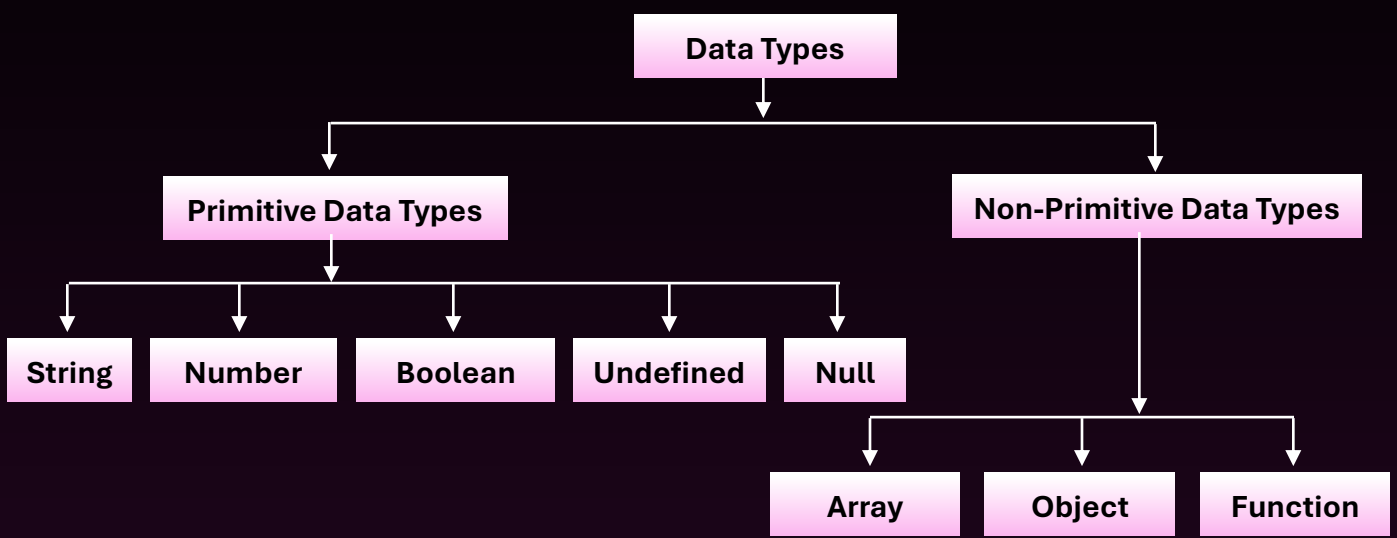
A data type defines the kind of value a variable can hold like a number, text, or true/false. JavaScript automatically detects the data type based on the value assigned to a variable.

ডেটা টাইপ বলতে বোঝায়, একটি variable কী ধরনের value ধারণ করতে পারে যেমন number, text, বা true/false। একটি variable এ কী value দেওয়া হয়েছে তার উপর ভিত্তি করে JavaScript automatically সেই data type চিনে নেয়।

Types of Data Types

JavaScript data types are divided into two main categories: **primitive** and **non-primitive**.

জাভাস্ক্রিপ্ট ডেটা টাইপকে প্রধানত দুই ভাগে ভাগ করা হয়: primitive এবং non-primitive।



Primitive Data Types

Primitive data types store a single, simple value. They are **immutable**, meaning the value itself cannot be changed once created.

Primitive ডেটা টাইপ একটি মাত্র সরল value ধারণ করে। এগুলো immutable, অর্থাৎ একবার তৈরি হলে value নিজে পরিবর্তন করা যায় না।

```
let name = "Y";         // String
let age = 25;           // Number
let isStudent = true;   // Boolean
```

Non-Primitive Data Types

Non-primitive data types can store multiple values or complex data in a single variable. They are **mutable**, meaning their content can be changed after creation.

Non-primitive ডেটা টাইপ একটি variable-এ একাধিক value বা জটিল data ধারণ করতে পারে। এগুলো mutable, অর্থাৎ তৈরির পরেও এদের ভেতরের content পরিবর্তন করা যায়।

```
let colors = ["red", "green", "blue"]; //Array
let user =
{ name: "X",
  age: 52
}; //Object
```

Primitive Data Types

JavaScript has **seven primitive** data types. Let's go through them one by one.

জাভাস্ক্রিপ্টে সাতটি primitive ডেটা টাইপ আছে। চলো একটা একটা করে দেখা যাক।

String

A string is a **sequence of characters** used to store text. It is written inside single quotes, double quotes, or backticks.

বাংলা: String হলো অক্ষরের একটি ক্রম, যা text সংরক্ষণ করতে ব্যবহৃত হয়। এটি single quote, double quote অথবা backtick-এর ভেতরে লেখা হয়।

Why is it called a "String"?

The word "string" means a thread. In programming, it is used as an analogy just like beads are strung one after another on a thread, characters (letters, numbers, symbols) are arranged one by one in order to form a string.

"String" শব্দের অর্থ সুতা। সুতায় যেমন একটার পর একটা পুঁতি গাঁথা থাকে, তেমনি string-এ অক্ষর, সংখ্যা বা চিহ্ন (character) একটার পর একটা সাজানো থাকে।

let word = "HELLO";

H → E → L → L → O

প্রতিটা character-এর একটা position (index) থাকে, যা **0 থেকে** শুরু হয়, ঠিক সুতার প্রথম পুঁতি থেকে গুনে যাওয়ার মতো।

Quotes in Strings

A string can be written using single quotes (' '), double quotes (" "), or backticks (` `). Single and double quotes work the same way, so you can use either one, but the opening and closing quote must match. Backticks are special, **they allow you to insert variables or expressions directly inside the text using ${ }, and they also support multi-line strings.**

String লেখার জন্য single quote (' '), double quote (" ") অথবা backtick (` `) ব্যবহার করা যায়। Single ও double quote একইভাবে কাজ করে, তাই যেকোনোটি ব্যবহার করা যায়, তবে শুরু ও শেষের quote একই হতে হবে। Backtick একটু বিশেষ: এর ভেতরে ${ } ব্যবহার করে সরাসরি variable বা expression বসানো যায়, এবং একাধিক লাইনের string-ও লেখা যায়।

```
let city = 'Chattogram';      "Chattogram": Unknown word.
console.log(city); //Chattogram     "Chattogram": Unknown word.
let userName = "Z";
console.log(userName); //Z
let Studentname = "W";     "Studentname": Unknown word.
let greeting = `Hello, ${Studentname}!`;     "Studentname": Unknown word.
console.log(greeting); // Hello, W!
```

String Length

The length property returns the total number of characters in a string, including letters, numbers, spaces, and symbols.

length property একটি string-এ মোট কতটি character আছে তা জানায়। এতে অক্ষর, সংখ্যা, space এবং symbol সবই গণনা করা হয়।

```
let cstmrName = "idk";     "cstmr": Unknown word.
console.log(cstmrName.length);    // 6     "cstmr": Unknown word.

let text = "Hello World";
console.log(text.length);    // 11 (Spaces are also counted)

let empty = "";
console.log(empty.length);  // 0
```

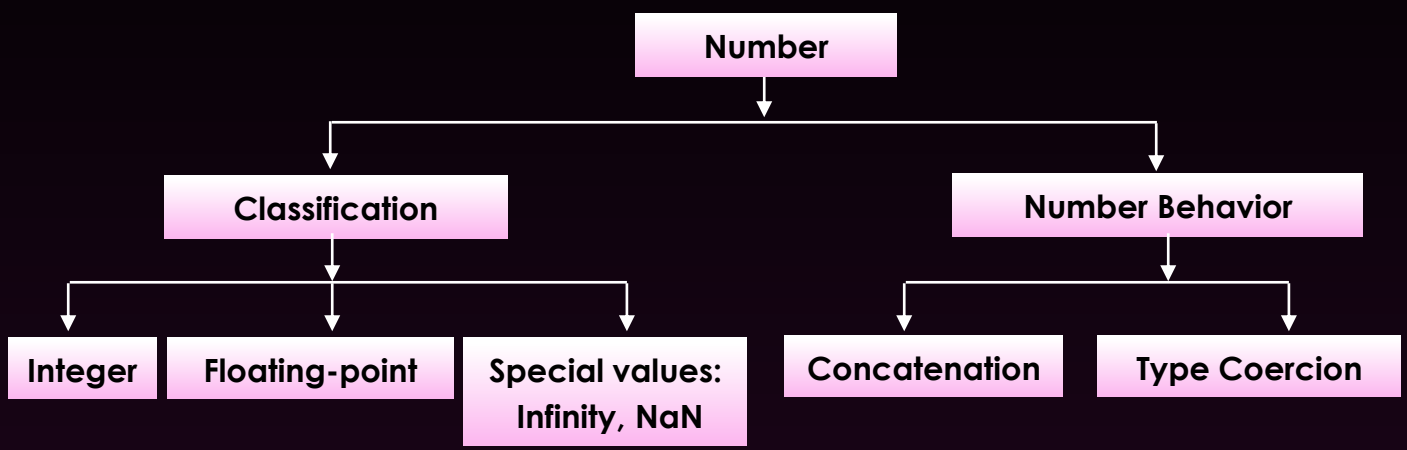
Number

A number stores both **integers** and **decimal values**. JavaScript uses a single type for all numbers.

Number পূর্ণসংখ্যা (integer) এবং দশমিক সংখ্যা (decimal) দুটোই সংরক্ষণ করে। JavaScript-এ সব ধরনের সংখ্যার জন্য একটিই type ব্যবহৃত হয়।

```
let age_ = 25;          // integer
let price = 99.5;       // decimal

console.log(age_ + 5);        // 30
console.log(price * 2);       // 199
console.log(typeof age_);     // number
```



NaN (Not a Number)

NaN is a special number value that represents an invalid or failed mathematical operation. Although it means "Not a Number", its type is still number.

NaN হলো একটি বিশেষ number value, যা কোনো ভুল বা ব্যর্থ গাণিতিক operation-এর ফলাফল বোঝায়। নাম "Not a Number" হলেও এর type আসলে number।

```
let x = "abc" * 2;
let y = 25;
console.log(isNaN(x));   // true
console.log(isNaN(y));   // false
console.log(x === x);    // false (NaN is not equal to itself)
```

Key points:

- Trying to calculate with or convert a non-numeric value into a number results in NaN
  কোনো সংখ্যা নয় এমন value নিয়ে হিসাব করতে গেলে বা সেটিকে number-এ রূপান্তর করতে গেলে NaN পাওয়া যায়।
- NaN is not equal to itself, so use isNaN() instead of === to check for it
  NaN নিজের সাথেও সমান হয় না, তাই এটি চেক করতে === নয়, isNaN() ব্যবহার করতে হয়।

Concatenation

When the + operator is used between a number and a string, JavaScript joins them as text instead of adding them.

+ operator যখন number ও string-এর মধ্যে ব্যবহার করা হয়, তখন JavaScript যোগ না করে দুটিকে জুড়ে একটি string বানায়।

```
console.log(a + b);                 // 10 (number + number = addition)
console.log(c + a);                 // "55" (string + number = joined as text)
console.log(plantName + age);            // "Coin-Plan1"
console.log("Age: " + age);         // "Age: 1"
```

Type Coercion

Type coercion is JavaScript's automatic conversion of one data type into another when different types are used together in an operation.

বাংলা: Type coercion হলো বিভিন্ন type একসাথে ব্যবহার করলে JavaScript-এর নিজে থেকে এক data type-কে অন্য type-এ রূপান্তর করা।

```
let x = "10";
let y = 5;
let isTrue = true;

console.log(x - y);           // 5 (string → number, subtraction works)
console.log(x * y);           // 50 (string → number, multiplication works)
console.log(x + y);           // "105" (number → string, joined as text)
console.log(isTrue + y);      // 6 (true → 1)
```

Note: + string পেলে জোড়া লাগায়, কিন্তু –, *, / string-কে number বানিয়ে হিসাব করে।

Boolean

A boolean has only two possible values, true or false. It is mostly used in conditions and decision making.

Boolean-এর মাত্র দুটি সম্ভাব্য value থাকে, true অথবা false। এটি বেশিরভাগ ক্ষেত্রে condition ও সিদ্ধান্ত নেওয়ার কাজে ব্যবহৃত হয়।

Key Boolean Characteristics

- true and false are boolean data types
- true and false are the only possible boolean values
- true and false must be written in **lowercase**
- true and false must be written **without quotes**

Boolean-এর মূল বৈশিষ্ট্য

- true এবং false হলো boolean data type
- true এবং false ই boolean-এর একমাত্র সম্ভাব্য value
- true এবং false অবশ্যই ছোট হাতের অক্ষরে (lowercase) লিখতে হয়
- true এবং false অবশ্যই quote ছাড়া লিখতে হয়

Boolean Use Cases

Very often, in programming, you will need a data type that can represent one of two values, like

- yes or no
- on or off
- true or false

Boolean-এর ব্যবহার

প্রোগ্রামিং-এ প্রায়ই এমন একটি data type দরকার হয়, যা দুটি value-এর মধ্যে একটিকে উপস্থাপন করতে পারে, যেমন:

- হ্যাঁ (yes) অথবা না (no)
- চালু (on) অথবা বন্ধ (off)
- সত্য (true) অথবা মিথ্যা (false)

```
let a = true;       // boolean
let b = "true";     // string
console.log(typeof a);   // boolean
console.log(typeof b);   // string
```

Undefined

A variable that has been declared but not assigned any value automatically gets the value undefined.

কোনো variable declare করা হয়েছে কিন্তু তাতে কোনো value assign করা হয়নি, তখন সেটি automatically undefined value পায়।

```
let score;
console.log(score);          // undefined
console.log(typeof score);   // undefined
```

Null

Null represents an intentional empty value. It is assigned manually to show that a variable has no value on purpose.

Null হলো ইচ্ছাকৃতভাবে রাখা একটি খালি value। কোনো variable-এ ইচ্ছা করেই "কিছু নেই" বোঝাতে এটি নিজে assign করা হয়।

```
let result = null;
console.log(result);         // null
console.log(typeof result);  // object
```

Why is typeof null "object"?

typeof null returns "object" because of a bug in the first version of JavaScript. null is actually a primitive value, and the bug was never fixed to avoid breaking existing websites.

typeof null "object" দেখায় JavaScript-এর প্রথম version-এর একটি bug-এর কারণে। null আসলে একটি primitive value, এবং পুরনো website ভেঙে যাওয়ার ভয়ে এই bug কখনো ঠিক করা হয়নি।

Non-Primitive Data Types

Array

An array is an ordered **list of values stored in a single variable**. Each value has an index number, and the index always starts from 0.

Array হলো একটি variable-এ ক্রমানুসারে সাজানো একাধিক value-র তালিকা। প্রতিটি value-র একটি index নম্বর থাকে, এবং index সবসময় 0 থেকে শুরু হয়।

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]);        // Apple
console.log(fruits[2]);        // Mango
console.log(fruits.length);    // 3
```

Object

An object stores data as **key-value pairs**. Each key (also called a property name) is linked to a value, which makes it ideal for describing a real-world thing like a person or a product.

Object ডেটাকে key-value জোড়া হিসেবে সংরক্ষণ করে। প্রতিটি key (property-র নাম) একটি value-র সাথে যুক্ত থাকে, তাই মানুষ বা পণ্যের মতো বাস্তব জিনিস বর্ণনা করতে এটি খুবই উপযোগী।

```
let useR = {
  name: "R",        // key: name, value: "R"
  age: 25,          // key: age, value: 25
  isStudent: true   // key: isStudent, value: true
};
// Accessing values using dot notation
console.log(useR.name);        // R
// Accessing values using bracket notation
console.log(useR["age"]);      // 25
```

Array and Object

Arrays store ordered collections of values accessed by index, while objects store data as key-value pairs accessed by property name. Both are mutable, reference-based structures and are commonly combined to model real-world data.

Array index-এর মাধ্যমে ক্রমানুসারে value সংরক্ষণ করে, আর Object property-র নাম দিয়ে key-value জোড়া হিসেবে ডেটা সংরক্ষণ করে। উভয়ই mutable ও reference-ভিত্তিক এবং বাস্তব ডেটা মডেল করতে প্রায়ই একসাথে ব্যবহৃত হয়।